

Evolution of Benchmark: Black-Box Optimization Benchmark Design through Large Language Model

Chen Wang^{1*} Sijie Ma^{1*} Zeyuan Ma^{1*} Yue-Jiao Gong¹

Abstract

Benchmark Design in Black-Box Optimization (BBO) is a fundamental yet open-ended topic. Early BBO benchmarks are predominantly human-crafted, introducing expert bias and constraining diversity. Automating this design process can relieve the human-in-the-loop burden while enhancing diversity and objectivity. We propose Evolution of Benchmark (EoB), an automated BBO benchmark designer empowered by the large language model (LLM) and its program evolution capability. Specifically, we formulate benchmark design as a bi-objective optimization problem towards maximizing (i) landscape similarity to target tasks and (ii) algorithm-differentiation ability across a portfolio of BBO solvers. Under this paradigm, EoB iteratively prompts LLM to evolve a population of benchmark programs and employs a reflection-based scheme to co-evolve the landscape and its corresponding program. Comprehensive experiments validate our EoB is a competitive candidate in multi-dimensional usages: 1) Benchmarking BBO algorithms; 2) Training and testing learning-assisted BBO algorithms; 3) Extending proxy for expensive real-world problems.

1. Introduction

Black-Box Optimization (BBO) problems are inherently hard (Jones et al., 1998) and widely encountered across various real-world scenarios (Gen & Cheng, 1999; Terayama et al., 2021; Ma et al., 2025e). During the past two decades, a broad spectrum of BBO algorithms (Holland, 1992; Storn & Price, 1997; Beyer & Schwefel, 2002; Kennedy & Eberhart, 1995; Snoek et al., 2012; Byrd et al., 2016; Akiba et al., 2019; Golovin et al., 2017) have been proposed to address

^{*}Equal contribution ¹South China University of Technology. Correspondence to: Yue-Jiao Gong <gongyuejiao@gmail.com>.

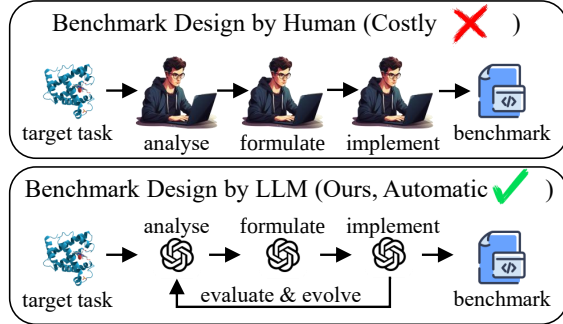


Figure 1. Paradigm shift from human-crafted benchmark design toward LLM-based automation.

BBO problems (Holland, 1992; Storn & Price, 1997; Beyer & Schwefel, 2002; Kennedy & Eberhart, 1995; Snoek et al., 2012; Byrd et al., 2016). However, compared to the exponential growth in algorithm development, the advancement of BBO benchmarks has lagged significantly behind.

A well-developed BBO benchmark is crucial for the advancement of the BBO field, as it helps ensure both fair comparison and performance coherence on out-of-benchmark tasks. Existing benchmarks such as CoCo-BBOB (Hansen et al., 2010) and IEEE CEC series (Liang et al., 2014) are commonly used within the community, where the synthetic functions are carefully designed to provide various landscape challenges (Mersmann et al., 2011) for evaluating BBO algorithms. However, their limited testing instances hinder them from keeping pace with the swift emergence of novel BBO tasks in this fast-evolving era. Besides, given the rise of learning-assisted BBO techniques more recently (Ma et al., 2025d; van Stein & Bäck, 2024b; Li et al., 2024b), training them requires a large number of diversified BBO instances to ensure robust generalization. This poses significant challenges for hand-crafted benchmark design since this paradigm needs deep expertise and massive labor input.

To address these fundamental limitations, we propose Evolution of Benchmark (EoB) as a compact, self-contained and fully automated BBO benchmark designer. As illustrated in Fig. 1, EoB is empowered by recent advances of LLMs for program evolution (Guo et al., 2023; Liu et al., 2024; van Stein & Bäck, 2024a; Novikov et al., 2025) to achieve both benchmark quality and automated design ef-

efficiency. In such a paradigm, a population of feasible programs is maintained and undergoes an iterative code-level reproduction process. Those programs with better fitness are selected and kept until the end of evolution. However, unlike the heuristic search focused in prior works, BBO benchmark design presents unique challenges that require specialized solutions: 1) **Complex design objectives alignment**: We introduce a multi-objective program evolution paradigm that simultaneously optimizes the landscape similarity and algorithm distinguishing capability of benchmarks. 2) **Program-landscape co-evolution**: We propose a novel reflection-based evolution strategy that leverages LLM’s deep understanding of mathematical structures to achieve program-landscape co-evolution. By analyzing the causal relationship between program patterns and performance, this strategy enables the automatic synthesis of complex benchmark functions without explicit landscape editing.

We summarize the contributions of this paper in threefold:

- EoB represents the paradigm shift from human-crafted to LLM-automated BBO benchmark design, delivering orders of magnitude efficiency improvement while maintaining benchmarking effectiveness.
- Our key innovations include (i) the bi-objective program search modeling, (ii) the tailored LLM-based program evolution paradigm, and (iii) the reflective landscape editing scheme, which ensure the automation and performance of the search process.
- EoB undergoes rigorous empirical validation across different perspectives: classic BBO algorithm benchmarking, modern learning-assisted optimizer training, and proxy modeling for expensive real-world problems, which underscores its universal applicability and practical value for the BBO community.

2. Related Works

2.1. BBO Benchmarks

Compared to the exponential development scale of BBO algorithms, exploration and in-depth analysis of BBO benchmarks are still far from sufficient. Initial benchmark-related efforts include synthetic suites with closed-form functions, such as CoCo-BBOB (Hansen et al., 2010) and IEEE CEC Benchmark suites (Mohamed et al., 2021), as well as framework-level tools like IOHProfiler (Doerr et al., 2019), which supports benchmarking on these suites. They are simple and easy to analyze, hence being commonly used in the community. Further efforts include category extensions (Tian et al., 2017; Wu et al., 2017; Da et al., 2017; Zhong et al., 2024), reproducible baselines (Duan et al., 2024), and efficiency optimization (Huang et al., 2024).

However, recent advances in learning-assisted BBO algorithms (Ma et al., 2023b; 2025c) reveal these synthetic benchmarks show limited performance evaluation coherence due to the gap between realistic tasks and these synthetic toy problems. Prior attempts to address this include hand-crafted extensions (Hwang et al., 2010; Arango et al., 2021; Shehadeh & Kudela, 2025; Xue et al., 2025), genetic programming-based search (Muñoz & Smith-Miles, 2020; Long et al., 2023; Tian et al., 2020; Wang et al., 2025), and affine recombination across CoCo-BBOB instances to automatically create new test functions by filling gaps in the instance space (Dietrich & Mersmann, 2022; Vermetten et al., 2023). However, hand-crafted extensions are both time- and resource-consuming, while the latter two approaches depend on deep expertise in domain-specific language. In contrast, Our EoB uses LLM-based program evolution to design benchmarks automatically and efficiently with minimal need for deep expertise.

2.2. Program Evolution by LLMs

We briefly review representative LLM-based program evolution methods. The core idea is that: by integrating LLMs with an evaluator to provide proper performance feedback, they might gain strength in open discovery through iterative in-context learning (Min et al., 2022). The early efforts in the field includes FunSearch (Romera-Paredes et al., 2024) and EoH (Liu et al., 2024) for heuristic discovery, EvoPrompting (Chen et al., 2023) for neural architecture search and EvoPrompt (Guo et al., 2023) for automatic prompt optimization. They share an evolutionary paradigm where a population of feasible programs are maintained and go through tailored reproduce-then-select loop until an optimal program is evolved. Following these initial examples, a wide array of researches are emerged (Ye et al., 2024; van Stein & Bäck, 2024a; Yao et al., 2025; Liu et al., 2025b; Novikov et al., 2025; Ma et al., 2023a). Such automation paradigm has also been extended to benchmark design. LLM-EBG (Ono et al., 2025) evolves high-performance function code by optimizing the performance gap between two optimizers, while Skvorc et al. (Skvorc et al., 2026) use LLaMEA (van Stein & Bäck, 2024a) and a pre-trained feature predictor to customize instances toward desired landscape properties. Our EoB follows the same core but differs in three key aspects: 1) multi-objective benchmark evolution for practical design needs; 2) two validated benchmark-generation objectives; and 3) the first use of LLM-generated benchmarks as training distributions for learning-assisted optimizers.

3. Methodology

3.1. Overview

Given M design requirements (objectives) $\mathcal{O}_1, \dots, \mathcal{O}_M$, such as the landscape diversity (Muñoz et al., 2014) and

algorithm distinguishing capability (Kostovska et al., 2023), we formulate the benchmark design problem as a multi-objective optimization problem:

$$\min_{f \in \mathbb{F}} (\mathcal{O}_1(f), \dots, \mathcal{O}_M(f)), \quad (1)$$

where $\mathbb{F}$ spans a comprehensive function space, which in this paper are all feasible programs that could be used as objective functions for optimization problems. The M design objectives are probably conflict with each other, hence we obtain a Pareto front set $\{f_1^*, \dots, f_K^*\}$ with K non-dominated program instances, which can be regarded as a benchmark suite with multiple diverse testing instances. At the formulation level, EoB shares some similarity with ISA (Muñoz & Smith-Miles, 2017). Unlike ISA, EoB uses this formulation for benchmark generation rather than performance analysis. For the basics of Pareto and domination, refer to (Deb et al., 2016).

3.2. EoB

EoB treats each benchmark function as an evolving program, searching through the vast space of possible mathematical formulations using a multi-objective evolutionary strategy in MOEA/D (Zhang & Li, 2007) manner. It operates through five interconnected stages: initialization creates a diverse population of candidate functions; evaluation measures each function’s landscape characteristics and algorithm discrimination power; evolution employs LLM-driven reflection to generate improved variants; selection maintains evolutionary pressure through Pareto-based comparison; and archive management preserves the best discoveries.

3.2.1. INITIALIZATION

The initialization in EoB resembles that of MOEA/D. First, we generate a group of reference vectors uniformly spread in the bi-objective space. The reference vectors are N 2-dimensional ones:

$$\vec{\lambda}_i = \left(\frac{i}{N-1}, \frac{N-1-i}{N-1} \right), \quad i = 0, 1, 2, \dots, N-1. \quad (2)$$

We then randomly sample a function program $f_i^{t=0}$ from LLM for each $\vec{\lambda}_i$, resulting in an initial solution population of size N , where t denotes the evolutionary generation. To sample the program solutions from LLM, we repeatedly call an existing LLM using a carefully constructed *Init_Prompt* (see Appendix A). The aim of this prompt is, on the one hand, to help LLM grasp the basic context information, such as its role (an optimization expert and benchmark designer) and its task (to generate a program of an objective function). On the other hand, it should steer the LLM with sufficient expert-level specifications, which include two major parts.

Generation Preference. Given the bi-objective setting in EoB, we add an additional paragraph in *Init_Prompt* to inform the LLM which reference vector $\vec{\lambda}_i$ it generates a function program for. For example, when $\vec{\lambda}_i = (1, 0)$, the LLM is instructed to generate a function program that considers maximizing its similarity with the target problem rather than its algorithm distinguishing capability.

Population Diversity. If we do not explicitly ask the LLM to generate function programs with different landscape characteristics, it tends to generate highly similar instances, which greatly harms the diversity in the initial population. To this end, we have summarized seven kinds of landscape construction knowledge from existing literature (Yazdani et al., 2025; Cenikj et al., 2026; Seiler et al., 2025; Mersmann et al., 2011), and added them into our prompt to steer the initialization. We list three examples here and provide details of all seven types of knowledge in Appendix A.2: 1) *Asymmetric masking*: LLMs can use masking functions (e.g., $\text{np.where}(x \leq b, \cdot, \cdot)$) to construct asymmetric objective landscapes; 2) *Nonlinearity composition*: LLMs can use nested nonlinear operations (e.g., $\frac{1}{\text{abs}(\tanh(x))+1}$) to create highly twisted landscapes; 3) *Polynomial periodicity*: LLMs can insert polynomial terms into periodic functions (e.g., $\sin(2\pi x^3)$) to create non-stationary periodic oscillations. During the initialization phase, we randomly select one of the seven types of landscape construction knowledge and add the selected one into the *Init_Prompt*, ensuring the initialized population comprises programs with diverse landscape characteristics.

3.2.2. EVALUATION

While our EoB supports flexible design requirements (as shown in Eq. 1), we address two major objectives that are essential for BBO benchmark design in this paper: $\mathcal{O}_1$, Landscape Similarity Indicator (LSI), which indicates whether a searched program shows landscape similarity with the target hard-to-model BBO tasks; and $\mathcal{O}_2$, Algorithm Distinguishing Capability (ADC), which measures how effectively the searched program differentiates various BBO algorithms. Both objectives should be maximized.

LSI. Given a group of H optimization tasks of interest $F_{\text{target}} : \{p_1, \dots, p_H\}$, a primary objective is to design a group of K synthetic testing instances $F : \{f_1, \dots, f_K\}$ that exhibit similar landscape features to problems in F_{target} . Thus, we derive a similarity checking metric, LSI, to measure how similar a testing program $f \in \mathbb{F}$ designed by EoB is to F_{target} . Specifically, we first compute the landscape features of either problems in F_{target} or the testing program f through exploratory landscape analysis (ELA) (Mersmann et al., 2011), which samples a set of solutions and corresponding objective values to compute diverse landscape features, such as convexity, for an optimization problem. In

stead of directly using the computationally inefficient ELA features, we adopt NeurELA (Ma et al., 2025b), a recently proposed neural network-based landscape profiling system. For each $p \in F_{target}$, we use NeurELA to obtain their 16-dimensional feature vector z_p . For f , we obtain z_f . Then, the LSI of f under F_{target} is formulated as:

$$\mathcal{O}_1(f|F_{target}) = \frac{1}{1 + \frac{1}{\sigma^2} \min_{p \in F_{target}} \|z_f - z_p\|_2}, \quad (3)$$

where σ is a self-adaptive scale factor computed from the distribution of F_{target} in the NeurELA feature space:

$$\sigma = \frac{1}{H} \sum_{i=1}^H \min_{j \neq i} \|z_{p_i} - z_{p_j}\|_2, \quad (4)$$

where $z_{p_i} \in \mathbb{R}^{16}$ denotes the NeurELA feature vector of target instance p_i . Here, σ normalizes the feature-space distance such that LSI remains comparable across target task sets with different scales. A larger $\mathcal{O}_1$ indicates that the program f designed by LLM is at least similar to a problem instance $p \in F_{target}$. This metric, by encouraging generated functions to align with various established landscape features, partly contributes to the overall diversity of the evolved benchmark suite. Further, by additionally considering the following ADC metric, EoB explores a set of solutions (benchmark programs) along the Pareto front, which exhibit good diversity.

ADC. The second benchmark design objective we consider in EoB is its algorithm distinguishing capability. Suppose we have a testing program f designed by EoB and a BBO algorithm pool $\Lambda : \{A_1, \dots, A_L\}$ with L BBO algorithms. ADC measures performance variation across these BBO algorithms on f by first running the L algorithms on f for B random seed-controlled independent runs. We record a $B \times L$ -dimensional objective matrix Obj where each entry $Obj_{i,j}$ records the best-so-far objective value achieved by A_j while optimizing f at the i -th independent run. Then the ADC of f over the algorithm pool Λ is formulated as:

$$\mathcal{O}_2(f|\Lambda) = \text{std}\left(\frac{1}{B} \sum_{i=1}^B \vec{Obj}_{i,:}\right), \quad (5)$$

where $\vec{Obj}_{i,:}$ denotes the best-so-far objective values achieved by all BBO algorithms in Λ at the i -th run. We also note that to address the variation in objective values across different f designed by EoB, we apply min-max normalization on the Obj matrix. A larger ADC indicates that f is sufficiently good and challenging for distinguishing diverse BBO algorithms in terms of their performances. This focus on algorithm performance is different from directly measuring landscape diversity, since there is no direct relationship between the two. In EoB, landscape diversity emerges indirectly from the Pareto trade-off between LSI

and ADC, where generated functions are pushed toward different balances of landscape similarity with target BBO tasks and distinguishing capability across BBO algorithms. For benchmarking purposes, ADC is the more direct criterion for revealing performance differences among optimizers.

PBI. For each individual program f_i in the initialized population and subsequent evolving populations, we first compute its LSI $\mathcal{O}_1(f)$ and ADC $\mathcal{O}_2(f)$. After objective computation, we further compute Penalty-based Boundary Intersection (PBI), a scalarization trick to turn multi-objective space into measurable scalar performance. PBI receives three inputs: $\mathcal{O}_1(f_i)$, $\mathcal{O}_2(f_i)$, and the reference vector $\vec{\lambda}_i$. It then outputs a scalar score that reflects the aggregated performance of f_i between the tradeoff of LSI ($\mathcal{O}_1$) and ADC ($\mathcal{O}_2$). The PBI score is used as reference information for subsequent LLM-based program evolution, including both the reproduction (Sec. 3.2.3) and selection (Sec. 3.2.4) phases. For details of PBI's computation, refer to the original MOEA/D paper (Zhang & Li, 2007).

3.2.3. EVOLUTION

Given a population of candidate programs $\{f_i^t\}_{i=1}^N$ at the t -th generation ($t = 0$ is the initialized population), we propose a two-stage reproduction scheme to fully leverage the LLM's outstanding reasoning capability for code-level understanding and program search.

Reflection stage. At the first stage, we iteratively prompt the LLM with pairs of candidate programs and instruct it to compare the objective scores of these program pairs. The LLM then engages in comparison and performs reflective thinking, resulting in valuable insights on how to improve the candidate programs. We provide a detailed prompt for this stage in Appendix A.3.1. Specifically, similar to MOEA/D, we first construct the neighborhood set for each f_i^t , which comprises 5 candidate programs with the closest Euclidean distance to f_i^t in terms of the corresponding reference vectors. Then, we randomly select two candidates f' and f'' from the neighborhood set. For each candidate program, the LLM is provided with three inputs: (i) Source code, (ii) scores of LSI, ADC, and PBI, and (iii) reference vectors. Considering that the candidate program is generated by the LLM, it may encounter syntax errors and other invalid issues; therefore, EoB steers the LLM to output comprehensive analyzes on a case-by-case basis:

- **Aggressive Mutation:** If f' and f'' are both invalid, the LLM is directed to analyze the reasons behind them and output corresponding suggestions to rectify the invalid issues or facilitate a complete regeneration.
- **Conservative Mutation:** If one of them is invalid, the LLM is directed to ignore the invalid one and focus on

the valid one to analyze why it works in LSI/ADC/PBI aspects and subsequently propose the corresponding improvement suggestions.

- **Reflection-Based Crossover:** If both of them are valid, the LLM is directed to conduct a three-fold in-depth reflection when comparing f' with f'' (where the candidate with the higher PBI is designated as the *winner* and the other as the *loser*): (i) Successful Pattern Identification: recognize the major code-level differences between *winner* and *loser*, summarize knowledge and its relationship with landscape characteristics. (ii) Beneficial Gene Preservation: if the *loser* shows a higher score in either LSI or ADC, the LLM is asked to analyze the reason behind it and preserve such valuable knowledge. (iii) Bi-objective Tradeoff Analysis: recognize and analyze the positive/negative impacts of certain parts of the programs' codes on LSI and ADC, with a special focus on the conflict case such as adding a mathematical operation increase LSI but harm ADC.

In the end of this stage, all intermediate analyzes and thoughts are summarized by the LLM and organized in a JSON-like response, including a summary of the overall reasoning process, condensed analysis results for the aforementioned case-by-case reflections, and suggestions/instructions for generating the next generation offspring program.

Reproduction stage. Given the JSON-like reflection obtained in the Reflection stage, the LLM is directed to refer to and follow these useful analysis or improvement suggestions to produce a single new program candidate f_i^{t+1} . By repeatedly calling this two-stage evolution scheme N times, we could obtain an updated candidate population. We provide detailed prompts for this stage at Appendix A.3.2. Such comparison-based reflection and construction could facilitate fine-grained and effective benchmark function discovery through the co-evolution of both code-level programs and corresponding landscape-level characteristics.

3.2.4. SELECTION

Given the parent population $\{f_i^t\}_{i=1}^N$ and the newly reproduced offspring population $\{f_i^{t+1}\}_{i=1}^N$, we first compare the PBI scores between f_i^t and f_i^{t+1} ; the one with the higher score survives. To further enhance exploration and exploitation in the evolutionary process, we additionally allow comparison between f_i^{t+1} and the neighbors of f_i^t . A neighboring parent program will be replaced by f_i^{t+1} if its PBI is lower. We restrict such neighboring selection operations to 2 times per individual to avoid premature convergence.

3.2.5. PARETO FRONT MAINTENANCE

EoB maintains the up-to-date Pareto front set along the iterative program evolution. In this work, we define Pareto dom-

Algorithm 1 Evolution of Benchmark (EoB)

Input: Large Language Model $\mathcal{LLM}$, Target BBO tasks F_{target} , BBO algorithm pool Λ , #Reference vectors N , #Evolution generations T , Pareto set $PF = \emptyset$.

Output: Finally obtained benchmark

- 1: Let $t = 0$, initialize population $\{f_i^t\}_{i=1}^N$ (Sec. 3.2.1);
 - 2: Evaluate each f_i^t (Sec. 3.2.2), update PF (Sec. 3.2.5);
 - 3: **while** $t < T$ **do**
 - 4: Reproduce offspring $\{f_i^{t+1}\}_{i=1}^N$ (Sec. 3.2.3);
 - 5: Evaluate each f_i^{t+1} (Sec. 3.2.2);
 - 6: Select elites from $\{f_i^t\}_{i=1}^N \cup \{f_i^{t+1}\}_{i=1}^N$ (Sec. 3.2.4);
 - 7: Update PF (Sec. 3.2.5) with elites, $t \leftarrow t + 1$;
 - 8: **end while**
 - 9: **return** PF
-

inance between two programs f_i and f_j as: f_i dominates f_j if and only if $\mathcal{O}_1(f_i) \geq \mathcal{O}_1(f_j)$ and $\mathcal{O}_2(f_i) \geq \mathcal{O}_2(f_j)$, and vice versa. A program is regarded as a Pareto front if all other programs in the population cannot dominate it. Initially, an empty Pareto front set PF is established. After initialization, PF is updated by the non-dominated programs in the population. At subsequent generations, PF is updated by first being concatenated with the selected non-dominated elite programs (from the union of parents and offspring), and then retaining those non-dominated ones.

After all designs have been introduced above, we present the overall workflow of EoB in Alg. 1, which shows a simple pipeline. After the evolution, each program in the Pareto front set can be regarded as a good benchmark function. The whole set can hence be regarded as a complete benchmark, robustly providing both desired landscape similarity to target tasks and effective algorithm distinguishing capabilities.

4. Experimental Results

In this section, we provide empirical validation to demonstrate 1) EoB provides an automated way to address multi-dimensional benchmarking needs in BBO community; 2) Each design proposed for EoB is effective. In the rest of this section, we first summarize basic experimental settings in Sec. 4.1 and then dive into the above two aspects in Sec. 4.2, Sec. 4.3 Sec. 4.4 and Sec. 4.5.

4.1. Basic Setup

EoB. The base LLM used in EoB is Deepseek-v3.2 (Liu et al., 2025a) with its default setting. For all of the following experiments, we set the number of reference vectors $N = 32$ (which is also the population size), the number of evolution generations $T = 10$. Each generated EoB function is evaluated in the default $D = 10$ setting, while D can be adjusted according to the dimensionality of the target BBO tasks. The evaluation budget for each generated

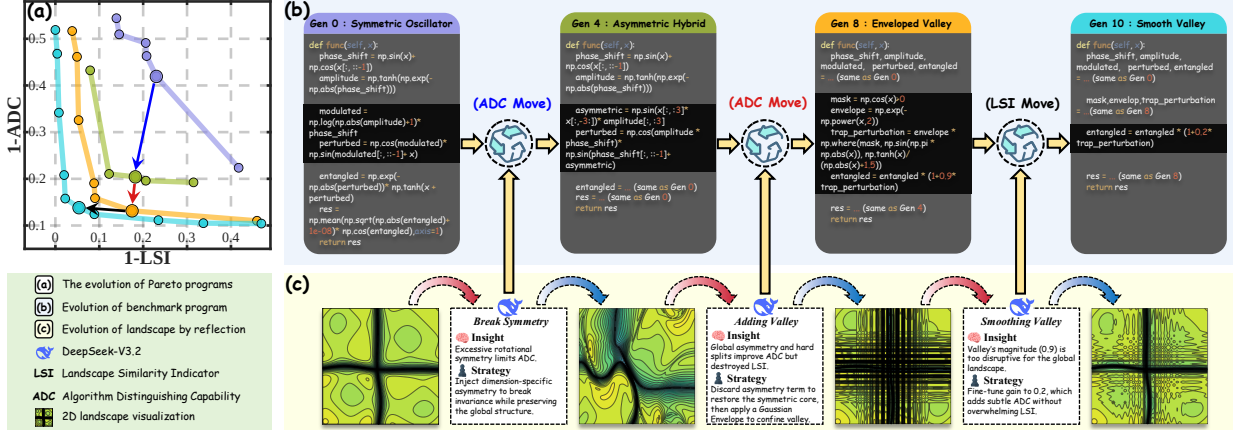


Figure 2. Visualization of EoB on the co-evolution of the benchmark program and corresponding landscape characteristics.

EoB function is set to $\max FEs = 20000$. For each F_{target} , H is determined by the number of instances in the corresponding target task set. As for the BBO algorithm pool Λ , we select from related literature 10 representative BBO algorithms: Vanilla DE (Storn & Price, 1997), JDE21 (Brest et al., 2021), MadDE (Biswas et al., 2021), NL-SHADE-LBC (Stanovov et al., 2022), PSO (Kennedy & Eberhart, 1995), GLPSO (Gong et al., 2015), sDMS-PSO (Liang et al., 2015), SAHLPSO (Tao et al., 2021), CMA-ES (Hansen et al., 2003), Sep-CMA-ES (Ros & Hansen, 2008). They cover diverse categories and show diverse optimization capabilities in our preliminary study, hence beneficial for measuring ADC score. We use these $L = 10$ algorithms and $B = 10$ independent runs to compute ADC scores. The weighted sum strength factor θ in PBI score computation is set to 5. At last, we note that the choice of target BBO tasks F_{target} varies in different experiments to demonstrate multi-dimensional usages of EoB, which we will detail in each experiment section.

Involved Benchmarks. Several benchmarks are involved to serve as both the compared baselines and the target BBO tasks F_{target} : 1) CoCo-BBOB (Hansen et al., 2010; 2021): which includes 24 hand-crafted synthetic functions with diverse landscape characteristics. It is the most representative BBO benchmark used by optimization researchers to evaluate and compare their BBO algorithms. 2) MetaBox (Ma et al., 2023b; 2025c), a benchmark platform proposed recently for learning-based BBO algorithms (Ma et al., 2025d). It contains a wide array of realistic benchmarks such as UAV path planning (Shehadeh & Kudela, 2025) (56 testing cases), hyper-parameter optimization (Arango et al., 2021) (935 testing cases). We use UAV, HPO to denote them. All experiments are conducted on a platform equipped with an RTX 2080Ti 11GB GPU, an Intel Xeon E5-2680 v4 @ 56x 3.3GHz CPU, and 128GB RAM. Using DeepSeek-v3.2, a complete evolutionary run takes approximately 20 minutes with a total API cost of \$0.37 (\$0.01

per generated function). We provide our project at <https://github.com/MetaEvo/EoB>.

4.2. Benchmarking BBO algorithms

One of the primary usages of EoB is to extend (construct) from representative benchmarks to enhance their benchmarking capability, such as the algorithm distinguishing capability. To this end, we set the target BBO tasks F_{target} as CoCo-BBOB benchmark, and run EoB (Alg. 1) multiple times to obtain 24 10-dimensional function programs¹ (the same size as CoCo-BBOB), which we name the searched benchmark as EoB-BBOB. We leave the detailed function programs in EoB-BBOB at our [project](#). We provide following interesting results to present how well EoB performs.

4.2.1. EVOLUTION EFFECTIVENESS

We first illustrate the evolution path of EoB’s program evolution process in Fig. 2. In the left of this figure, we plot the pareto front improvement during the evolution, which shows that the programs evolved by EoB continuously improve both the LSI and ADC scores along the program evolution, clearly demonstrating the effectiveness of EoB. We further delve into the details of such effective evolution in the right of this figure, where we showcase the concrete evolution path of a program individual (from its initialization to the end of search). The results there validate a key design motivation we mentioned before: by using LLM-based program evolution, EoB is capable of evolving the landscape of benchmark functions and its code-level implementation simultaneously. Driven by our proposed LLM-based bi-objective evolution, the function program is evolved step by step with clear and interpretable design thoughts. This significantly mitigates the expert-level knowledge and labor input required to design a benchmark.

¹One EoB run can not ensure the finally obtained PF contains 24 non-dominated programs, multiple runs (2-3) are needed.

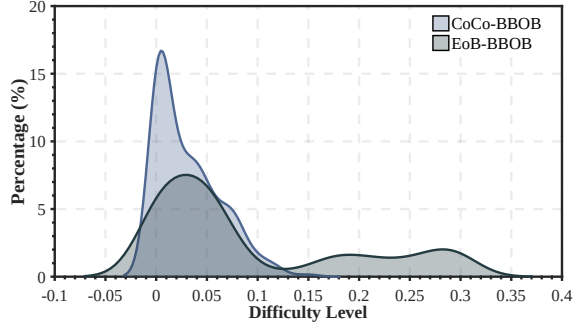


Figure 3. Benchmark diversity of CoCo-BBOB and EoB-BBOB.

4.2.2. BENCHMARK DIVERSITY COMPARISON

We further illustrate the instance-level diversity (more uniform is better) of CoCo-BBOB and the searched EoB-BBOB in Fig. 3. The results there are obtained by first testing the 10 algorithms in Λ on each of the 24 testing instances in a benchmark for 10 independent runs. For i -th testing instance and j -th run, we record the standard deviation of best-so-far objective values (min-max normalized) found by the 10 algorithms. A larger standard deviation indicates the testing instance is more challenging and hence easier to distinguish different BBO algorithms. Then, we obtain 240 standard deviation values which we use to plot the distribution density in Fig. 3. The results may indicate that: 1) Almost all hand-crafted synthetic functions in CoCo-BBOB show low algorithm distinguishing capability and difficulty diversity; 2) EoB could automatically design a more uniform and diversified benchmark by LLM-based program evolution, which is promising.

4.3. Training Learnable Optimizer

In this section we explore whether EoB could also address the new benchmarking needs in our community. More recently, learning-based BBO algorithms (Li et al., 2024a; Yang et al., 2025; Ma et al., 2025d; Kimiaei & Kungurtsev, 2025) emerge as an active and promising research avenue. They address the potential limitations in human-crafted BBO algorithms such as the adaptability issue and limited generalization by meta-learning (Finn et al., 2017; Thrun & Pratt, 1998) effective algorithm control policy that boosts classical BBO algorithms across a problem distribution. The finally learned BBO algorithms hence show preferred generalization towards unseen BBO tasks. Such paradigm gives birth to a large array of learnable optimizers (Guo et al., 2024; Zhao et al., 2024; Lange et al., 2023; Wu et al., 2025; Ma et al., 2025a; Gao et al., 2025; Du et al., 2025; Tian et al., 2025; Lu et al., 2025; Han et al., 2026; Pei et al., 2025). In this experiment, to ensure fast implementation and rigorous experimental protocol, we use four MetaBBO baselines well-maintained in MetaBox (Ma et al., 2023b; 2025c): LDE (Sun et al., 2021), GLEET (Ma et al., 2024),

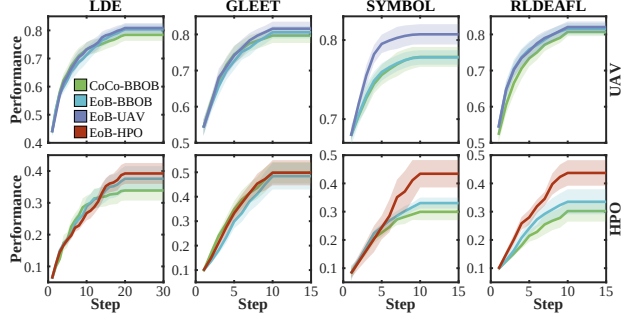


Figure 4. Zero-shot generalization performances of four learnable optimizers when trained by different benchmarks. Testing performance curves are presented.

SYMBOL (Chen et al., 2024) and RLDEAFL (Guo et al., 2025).

4.3.1. GENERALIZATION ADVANTAGE

The most important aspect cared about by the users of learnable optimizers is its true generalization potential. To validate EoB’s advantage on this aspect, we establish two validation scenarios: 1) A user trains learnable optimizer on a benchmark and then expects the trained model generalizes well on UAV tasks; 2) The same scenario yet the target tasks are HPO tasks. The benchmark baselines used for training in 1) are CoCo-BBOB, EoB-BBOB we searched in Sec 4.2 and EoB-UAV, which is searched by EoB using UAV-train-set² as the F_{target} in Alg. 1. And the baselines for 2) are CoCo-BBOB, EoB-BBOB and EoB-HPO (HPO-train-set as F_{target}). For the four learnable optimizer baselines LDE, GLEET, SYMBOL and RLDEAFL, we train them on the two scenarios by their default settings and the three benchmark baselines in the two scenarios respectively, and test the trained models on UAV-test-set and HPO-test-set for 10 independent optimization episodes. We plot the optimization curves in terms of generalization performances in Fig. 4, where the results clearly underscore that: 1) Through the bi-objective program evolution, the benchmark optimized by EoB could provide high-quality training source for learnable optimizers; 2) CoCo-BBOB, while regarded as a golden standard for evaluating classic BBO algorithms, may not be a proper choice for learning-assisted methods due to its biased human design. However, up to now, it is still the first choice in many learnable optimizers. The searched programs in EoB-UAV/HPO are at our [project](#).

We also want to emphasize the advantage of EoB in the efficiency side. Realistic BBO task such as UAV and HPO we discussed above are often quite expensive (either time-consuming or resource-consuming). However, training an advanced learnable optimizer require repeated interplay with

²Benchmarks in MetaBox are split into a train/test set to support learnable optimizers.

Table 1. The wall-clock evaluation time using EoB or target realistic tasks for MetaBBO approaches, less is better.

Algorithm	Scenario	UAV		HPO	
		Real	EoB	Real	EoB
LDE	Normal	30.38 min	3.124 s	23.72 min	2.923 s
	Large	≈ 7 h	28.32 s	≈ 4 h	27.43 s
GLEET	Normal	26.23 s	0.256 s	14.68 s	0.231 s
	Large	284.2 s	0.287 s	146.6 s	0.274 s
SYMBOL	Normal	43.68 s	0.423 s	190.8 s	0.412 s
	Large	78.95 s	0.528 s	312.1 s	0.547 s
RLDEAFI	Normal	98.24 s	0.921 s	15.98 s	0.852 s
	Large	22.91 min	1.442 s	205.7 s	1.352 s

Table 2. The final test-performance of GLHF trained by different baseline benchmarks.

Scenario	CoCo-BBOB	EoB-BBOB	EoB-Scenario
UAV	8.380E-01 ±5.193E-02	7.508E-01 ±1.149E-01	8.484E-01 ↑ ±3.854E-02
HPO	5.144E-01 ±2.113E-01	5.214E-01 ±1.769E-01	6.580E-01 ↑ ±1.933E-01

the target BBO tasks. If the target BBO tasks are expensive, training on them is impractical. To demonstrate this aspect, we present at Table 1 the average time consumed by the four learnable optimizer baselines per learning step when they use UAV/HPO as training set directly or use EoB-UAV/EoB-HPO instead. A significant efficiency leap could be observed from the results, which highlights an additional usage of EoB: searching for proxy benchmark for those expensive and hard-to-model BBO tasks.

4.3.2. GRADIENT-DEPENDENT CASE

Another interesting usage of EoB is when we consider a specific kind of learnable optimizer that requires the gradient information of the training problems to supervise its neural network model. An example is GLHF (Li et al., 2024b). This specific feature hinders such learnable optimizers from direct training on realistic BBO tasks where gradient information is not accessible, e.g., UAV/HPO. Fortunately, EoB could address this issue by searching executable function programs that show both high similarity (LSI) with the target tasks and substantial difficulty diversity (ADC), e.g., EoB-UAV/HPO. More importantly, they can be easily transformed into any autograd framework such as *torch* and *jax*, by LLM. The results in Table 2 validate EoB’s advantage for GLHF-like learnable optimizers.

4.3.3. KEY REASON BEHIND

At last, we give a simple yet intuitive interpretation on the reason behind EoB’s strength. Specifically, we first define a simple metric $\mathbb{C}(B_{source}, B_{target})$ that measures the performance consistency between a source benchmark B_{source} and a target benchmark B_{target} . Specifically, we first test the 10 BBO algorithms in our proposed algorithm pool Λ on B_{source} to obtain a 10-dimensional performance vector $\vec{S}_{source} : \{\frac{1}{rank_i}\}_{i=1}^{10}$, where $rank_i$ is the average

Table 3. Test-performance consistency between baseline benchmarks and realistic tasks.

Scenario	CoCo-BBOB	EoB-BBOB	EoB-Scenario
UAV	0.003	0.396	0.453 ↑
HPO	0.694	0.702	0.761 ↑

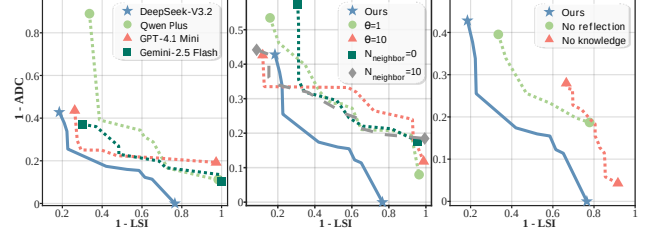


Figure 5. The ablation results shown as the pareto front set PF found by different baselines.

rank of i -th algorithm in Λ . We use the same logic to obtain $\vec{S}_{target}$. Then the performance consistency is computed as:

$$\mathbb{C}(B_{source}, B_{target}) = 1 - \mathcal{W}(\vec{S}_{source}, \vec{S}_{target}), \quad (6)$$

where $\mathcal{W}(\cdot, \cdot)$ denotes the Wasserstein distance. A larger consistency score indicates that the performance of BBO algorithms on the source benchmark is more consistent with the performance on the target task. Such performance predictivity may support the generalization of learnable optimizers. To validate this, we present in Table 3 the performance consistency between $\{\text{CoCo-BBOB}, \text{EoB-BBOB}, \text{EoB-UAV}, \text{EoB-HPO}\}$ and $\{\text{UAV}, \text{HPO}\}$. The results there, combined with the results in Fig. 4 and Table 2 cross-validate what we have assumed.

4.4. Ablation Studies

To validate the contribution of each component within EoB, we conducted three ablation studies: 1) LLM Backbone Capability, assessing the impact of the underlying model on evolutionary efficiency; 2) Hyperparameter Sensitivity, analyzing the effects of neighborhood size ($N_{neighbor}$) and PBI penalty (θ) on multi-objective optimization; and 3) Key Component Analysis, removing the Reflection Stage and Landscape Initialization Knowledge. Using the UAV-train set as the target task, the resulting Pareto fronts (Fig. 5) reveal that: 1) Deepseek-v3.2 achieves the most balanced performance among the backbones. 2) Diversity-oriented settings ($\theta = 10, N_{neighbor} = 10$) enhance LSI but compromise the objective trade-off, whereas exploitation-heavy settings ($\theta = 1, N_{neighbor} = 1$) suffer from low efficiency due to restricted information exchange. 3) The absence of Landscape Knowledge causes LSI stagnation, while removing the Reflection Stage degrades overall efficiency by severing the feedback loop of comparative insights.

We further conduct a landscape representation ablation to justify the use of NeurELA in LSI. Specifically, we replace

Table 4. Ablation on landscape feature representations in LSI. Values are final MetaBBO test performances, and larger is better.

Feature Representation	LDE	GLEET	SYMBOL	RLDEAFL
NeurELA	0.7621	0.8342	0.7428	0.8593
DeepELA	0.7509	0.7923	0.7295	0.7967
ELA: Meta-Model	0.7528	0.7685	0.7324	0.7891
ELA: Distribution	0.7424	0.7858	0.7188	0.7812
ELA: Convexity	0.7495	0.7612	0.7397	0.7954
ELA: Local Search	0.7487	0.7705	0.7242	0.7761
ELA: Levelset	0.7398	0.7902	0.7315	0.7905
ELA: Curvature	0.7462	0.7698	0.7332	0.7747

NeurELA with different traditional ELA feature sets (Mersmann et al., 2011) and DeepELA (Seiler et al., 2025), an attention-based feature extractor, while keeping other EoB settings unchanged. Since the available pre-trained DeepELA models are not dimension-free and cannot be directly applied to the higher-dimensional UAV tasks considered here, this ablation is conducted on the HPO setting following the training and testing protocol in Section 4.3. The results in Table 4 show that NeurELA achieves the best training effectiveness compared with both traditional ELA feature sets and DeepELA. Indeed, DeepELA shows competitive performance against the traditional ELA sets, but NeurELA is more capable in the EoB tasks considered here. This advantage may come from its learning-based pre-training, which leads to useful features that show correlation with traditional ELA and, more importantly, automatically filters out irrelevant features. Overall, these results support NeurELA as a practical and effective default feature extractor for LSI.

4.5. Parameter Sensitivity

We further analyze the sensitivity of EoB to three important settings, including the search dimension D , the population size N , and the number of target task instances H . For D , we construct a high-dimensional target task set using 100D CoCo-BBOB problem instances and set the search dimension of generated EoB functions to $D = 100$. The Pareto dynamics in Appendix B.1 (Fig. 11) show that EoB can progressively discover more balanced solutions with improved LSI and ADC, indicating that LLM-based functional synthesis still supports effective benchmark evolution in the 100D search space where manual function design is often less intuitive. For N , we compare $N = 8$, $N = 32$, and $N = 64$, and report the corresponding Pareto fronts in Appendix B.2 (Fig. 12). The setting $N = 8$ produces a relatively narrow Pareto front concentrated in the middle trade-off region, suggesting that a small population provides insufficient search diversity for jointly improving LSI and ADC. The setting $N = 64$ shows a trend similar to $N = 32$ and can obtain several better solutions, but it does not provide a clear overall advantage and inevitably introduces higher computational and time costs. Thus, $N = 32$ is adopted as a balanced setting between search effectiveness and computational efficiency. For H , we construct a larger target task set containing 240 CoCo-BBOB problem

instances, i.e., $H = 240$, and compare it with the default setting. The similar Pareto behavior in Appendix B.3 (Fig. 13) indicates that EoB is not sensitive to the setting of H .

5. Discussion

Takeaways. 1) If you are seeking a more challenging synthetic benchmark to test your BBO algorithms, feel free to use our EoB-BBOB; 2) If you are developing a learning-assisted BBO algorithm to solve some kind of expensive realistic tasks, run EoB to attain cheap synthetic benchmark to save your time and money; 3) You may consider using EoB to assist your analysis on some hard-to-model BBO tasks by extending their proxy.

Future Directions. We would like to discuss a few promising directions. The first is extending EoB towards diverse optimization categories, e.g., constrained optimization (Wu et al., 2017), multi-objective optimization (Zhang et al., 2008) and dynamic optimization (Zhong et al., 2024), for which EoB provides a universal and automated pipeline. Another important direction is to make the benchmark design objectives more fine-grained. While two design objectives we considered in this paper more or less provide an effective way to ensure comprehensive benchmarking, additional design objectives such as computational complexity (Huang et al., 2024) and surrogate difficulty (Ma et al., 2025f) could definitely help benchmarking in complex optimization tasks. At last, one may also consider using EoB’s paradigm for research fields such as AutoML (Gijssbers et al., 2024), AI4Science (Wei et al., 2025) and Benchmark of Benchmark (Qian et al., 2026), however, more efforts and certain expertise are still needed for adaption.

6. Conclusion

In this work, we address the bottleneck in benchmark design for the BBO field by replacing the laborious hand-crafted design process with an automatic paradigm. To this end, we propose EoB, the systematic framework to facilitate the evolution of high-quality benchmarks. EoB operates under a bi-objective LLM-based program search, aiming to discover evaluation programs (BBO instances) that simultaneously exhibit landscape similarity to target tasks and algorithm distinguishing capabilities. EoB is fully automatic through iterative multi-turn LLM conversations, ensuring minimal human-level design bias. Beyond its automation, EoB’s end-to-end paradigm supports diverse needs, including synthetic benchmark construction, training set design for learning-based BBO algorithms and proxy extension for hard-to-model BBO tasks. Ultimately, this work not only introduces a powerful new tool for automatic benchmark generation, but it also highlights the indispensable role of thoughtfully designed evaluation landscapes in advancing BBO research.

Acknowledgments

This work was supported in part by National Natural Science Foundation of China (Grant No. 62276100), in part by the Guangdong Provincial Natural Science Foundation for Outstanding Youth Team Project (Grant No. 2024B1515040010), in part by Guangzhou Science and Technology Elite Talent Leading Program for Basic and Applied Basic Research (Grant No. SL2024A04J01361), and in part by the Fundamental Research Funds for the Central Universities (Grant No. 2025ZYGXZR027).

Impact Statement

We claim a significant impact of our work on both research development in the optimization community and, more importantly, its foundational impact on how researchers and engineers approach the evaluation, refinement, and redesign of optimization algorithms. By automating the benchmark design process, the substantial labor resources and specialized expertise costs could be significantly reduced. Such an automated pipeline also helps mitigate human-based bias to assure benchmarking objectivity. This paradigm shift will significantly transform the evaluation and refinement of optimization algorithms, enabling more rigorous and objective comparisons that accelerate advancements across BBO and its diverse applications

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 2623–2631, 2019.
- Arango, S. P., Jomaa, H. S., Wistuba, M., and Grabocka, J. Hpo-b: A large-scale reproducible benchmark for black-box hpo based on openml. *arXiv preprint arXiv:2106.06257*, 2021.
- Beyer, H.-G. and Schwefel, H.-P. Evolution strategies—a comprehensive introduction. *Natural Computing*, 2002.
- Biswas, S., Saha, D., De, S., Cobb, A. D., Das, S., and Jalaian, B. A. Improving differential evolution through bayesian hyperparameter optimization. In *CEC*, 2021.
- Brest, J., Maučec, M. S., and Bošković, B. Self-adaptive differential evolution algorithm with population size reduction for single objective bound-constrained optimization: Algorithm j21. In *CEC*, 2021.
- Byrd, R. H., Hansen, S. L., Nocedal, J., and Singer, Y. A stochastic quasi-newton method for large-scale optimization. *SIAM Journal on Optimization*, 26(2):1008–1031, 2016.
- Cenikj, G., Nikolikj, A., Petelin, G., van Stein, N., Doerr, C., and Eftimov, T. A survey of features used for representing black-box single-objective continuous optimization. *Swarm and Evolutionary Computation*, 101: 102288, 2026.
- Chen, A., Dohan, D., and So, D. Evoprompting: Language models for code-level neural architecture search. *Advances in neural information processing systems*, 36: 7787–7817, 2023.
- Chen, J., Ma, Z., Guo, H., Ma, Y., Zhang, J., and Gong, Y.-j. Symbol: Generating flexible black-box optimizers through symbolic equation learning. In *ICLR*, 2024.
- Da, B., Ong, Y.-S., Feng, L., Qin, A. K., Gupta, A., Zhu, Z., Ting, C.-K., Tang, K., and Yao, X. Evolutionary multitasking for single-objective continuous optimization: Benchmark problems, performance metric, and baseline results. *arXiv preprint arXiv:1706.03470*, 2017.
- Deb, K., Sindhya, K., and Hakanen, J. Multi-objective optimization. In *Decision sciences*, pp. 161–200. CRC Press, 2016.
- Dietrich, K. and Mersmann, O. Increasing the diversity of benchmark function sets through affine recombination. In *International Conference on Parallel Problem Solving from Nature*, pp. 590–602. Springer, 2022.
- Doerr, C., Ye, F., Horesh, N., Wang, H., Shir, O. M., and Bäck, T. Benchmarking discrete optimization heuristics with iohprofiler. In *GECCO*, 2019.
- Du, Y., Yu, H., Xie, X., Zheng, Y., Zhan, L., Du, Y., Hu, C., Wang, B., and Jiang, J. Meta-black-box optimization with bi-space landscape analysis and dual-control mechanism for saea. *arXiv preprint arXiv:2511.15551*, 2025.
- Duan, Q., Zhou, G., Shao, C., Wang, Z., Feng, M., Huang, Y., Tan, Y., Yang, Y., Zhao, Q., and Shi, Y. Pypop7: A pure-python library for population-based black-box optimization. *Journal of Machine Learning Research*, 2024.
- Finn, C., Abbeel, P., and Levine, S. Model-agnostic meta-learning for fast adaptation of deep networks. In *ICML*, 2017.
- Gao, J., Liu, Y., Cheng, R., and Tan, K. C. Learning to evolve with convergence guarantee via neural unrolling. *arXiv preprint arXiv:2512.11453*, 2025.
- Gen, M. and Cheng, R. *Genetic algorithms and engineering optimization*. John Wiley & Sons, 1999.
- Gijsbers, P., Bueno, M. L., Coors, S., LeDell, E., Poirier, S., Thomas, J., Bischl, B., and Vanschoren, J. Amlb:

- an automl benchmark. *Journal of Machine Learning Research*, 25(101):1–65, 2024.
- Golovin, D., Solnik, B., Moitra, S., Kochanski, G., Karro, J., and Sculley, D. Google vizier: A service for black-box optimization. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pp. 1487–1495, 2017.
- Gong, Y.-J., Li, J.-J., Zhou, Y., Li, Y., Chung, H. S.-H., Shi, Y.-H., and Zhang, J. Genetic learning particle swarm optimization. *IEEE TC*, 2015.
- Guo, H., Ma, Y., Ma, Z., Chen, J., Zhang, X., Cao, Z., Zhang, J., and Gong, Y.-J. Deep reinforcement learning for dynamic algorithm selection: A proof-of-principle study on differential evolution. *TSMC*, 2024.
- Guo, H., Ma, S., Huang, Z., Hu, Y., Ma, Z., Zhang, X., and Gong, Y.-J. Reinforcement learning-based self-adaptive differential evolution through automated landscape feature learning. *arXiv preprint arXiv:2503.18061*, 2025.
- Guo, Q., Wang, R., Guo, J., Li, B., Song, K., Tan, X., Liu, G., Bian, J., and Yang, Y. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. *arXiv preprint arXiv:2309.08532*, 2023.
- Han, M., Li, X., Wu, K., Zhang, X., and Wang, H. Enhancing zero-shot black-box optimization via pretrained models with efficient population modeling, interaction, and stable gradient approximation. *Advances in Neural Information Processing Systems*, 38:33238–33274, 2026.
- Hansen, N., Müller, S. D., and Koumoutsakos, P. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (cma-es). *Evolutionary computation*, 2003.
- Hansen, N., Auger, A., Finck, S., and Ros, R. *Real-parameter black-box optimization benchmarking 2010: Experimental setup*. PhD thesis, INRIA, 2010.
- Hansen, N., Auger, A., Ros, R., Mersmann, O., Tušar, T., and Brockhoff, D. Coco: A platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 2021.
- Holland, J. H. *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. MIT press, 1992.
- Huang, B., Cheng, R., Li, Z., Jin, Y., and Tan, K. C. Evox: A distributed gpu-accelerated framework for scalable evolutionary computation. *IEEE Transactions on Evolutionary Computation*, 2024.
- Hwang, H., Vreven, T., Janin, J., and Weng, Z. Protein–protein docking benchmark version 4.0. *Proteins: Structure, Function, and Bioinformatics*, 2010.
- Jones, D. R., Schonlau, M., and Welch, W. J. Efficient global optimization of expensive black-box functions. *Journal of Global optimization*, 13(4):455–492, 1998.
- Kennedy, J. and Eberhart, R. Particle swarm optimization. In *Proceedings of ICNN’95-international conference on neural networks*, volume 4, pp. 1942–1948. iee, 1995.
- Kimiaei, M. and Kungurtsev, V. Machine learning algorithms for improving black box optimization solvers. *arXiv preprint arXiv:2509.25592*, 2025.
- Kostovska, A., Jankovic, A., Vermetten, D., Džeroski, S., Eftimov, T., and Doerr, C. Comparing algorithm selection approaches on black-box optimization problems. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, pp. 495–498, 2023.
- Lange, R. T., Schaul, T., Chen, Y., Zahavy, T., Dalibard, V., Lu, C., Singh, S., and Flennerhag, S. Discovering evolution strategies via meta-black-box optimization. In *The Eleventh International Conference on Learning Representations*, 2023.
- Li, P., Hao, J., Tang, H., Fu, X., Zhen, Y., and Tang, K. Bridging evolutionary algorithms and reinforcement learning: A comprehensive survey on hybrid algorithms. *IEEE Transactions on evolutionary computation*, 2024a.
- Li, X., Wu, K., Li, Y. B., Zhang, X., Wang, H., and Liu, J. Pretrained optimization model for zero-shot black box optimization. *Advances in Neural Information Processing Systems*, 37:14283–14324, 2024b.
- Liang, J., Qu, B., Suganthan, P., and Chen, Q. Problem definitions and evaluation criteria for the cec 2015 competition on learning-based real-parameter single objective optimization. *Technical Report 201411A*, Computational Intelligence Laboratory, Zhengzhou University, Zhengzhou China and Technical Report, Nanyang Technological University, Singapore, 29:625–640, 2014.
- Liang, J. J., Guo, L., Liu, R., and Qu, B.-Y. A self-adaptive dynamic particle swarm optimizer. In *2015 IEEE Congress on Evolutionary Computation (CEC)*, pp. 3206–3213. IEEE, 2015.
- Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025a.
- Liu, F., Tong, X., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., and Zhang, Q. Evolution of heuristics: Towards

- efficient automatic algorithm design using large language model. *arXiv preprint arXiv:2401.02051*, 2024.
- Liu, F., Lin, X., Yao, S., Wang, Z., Tong, X., Yuan, M., and Zhang, Q. Large language model for multiobjective evolutionary optimization. In *International Conference on Evolutionary Multi-Criterion Optimization*, pp. 178–191. Springer, 2025b.
- Long, F. X., Vermetten, D., Kononova, A. V., Kalkreuth, R., Yang, K., Bäck, T., and van Stein, N. Challenges of ela-guided function evolution using genetic programming. In *IJCCI*, 2023.
- Lu, C., Xue, K., Yuan, L., Wang, Y., Wang, Y., Sheng, F., and Qian, C. Sequential multi-agent dynamic algorithm configuration. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Ma, Y. J., Liang, W., Wang, G., Huang, D.-A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., and Anandkumar, A. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023a.
- Ma, Z., Guo, H., Chen, J., Li, Z., Peng, G., Gong, Y.-J., Ma, Y., and Cao, Z. Metabox: A benchmark platform for meta-black-box optimization with reinforcement learning. In *NeurIPS*, 2023b.
- Ma, Z., Chen, J., Guo, H., Ma, Y., and Gong, Y.-J. Auto-configuring exploration-exploitation tradeoff in evolutionary computation via deep reinforcement learning. In *GECCO*, 2024.
- Ma, Z., Cao, Z., Jiang, Z., Guo, H., and Gong, Y.-J. Meta-black-box-optimization through offline q-function learning. In *ICML*, 2025a.
- Ma, Z., Chen, J., Guo, H., and Gong, Y.-J. Neural exploratory landscape analysis for meta-black-box optimization. In *ICLR*, 2025b.
- Ma, Z., Gong, Y.-J., Guo, H., Qiu, W., Ma, S., Lian, H., Zhan, J., Chen, K., Wang, C., Huang, Z., et al. Metabox-v2: A unified benchmark platform for meta-black-box optimization. *arXiv preprint arXiv:2505.17745*, 2025c.
- Ma, Z., Guo, H., Gong, Y.-J., Zhang, J., and Tan, K. C. Toward automated algorithm design: A survey and practical guide to meta-black-box-optimization. *IEEE TEVC*, 2025d.
- Ma, Z., Huang, W., Song, G.-H., Guo, H., Ma, S., Cao, Z., and Gong, Y.-J. Evolutionary system 2 reasoning: An empirical proof. *arXiv preprint arXiv:2512.05760*, 2025e.
- Ma, Z., Huang, Z., Chen, J., Cao, Z., and Gong, Y.-J. Surrogate learning in meta-black-box optimization: A preliminary study. *arXiv preprint arXiv:2503.18060*, 2025f.
- Mersmann, O., Bischl, B., Trautmann, H., Preuss, M., Weihs, C., and Rudolph, G. Exploratory landscape analysis. In *Proceedings of the 13th annual conference on Genetic and evolutionary computation*, 2011.
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., and Zettlemoyer, L. Rethinking the role of demonstrations: What makes in-context learning work? *arXiv preprint arXiv:2202.12837*, 2022.
- Mohamed, A. W., Hadi, A. A., Mohamed, A. K., Agrawal, P., Kumar, A., and Suganthan, P. N. Problem definitions and evaluation criteria for the CEC 2021 special session and competition on single objective bound constrained numerical optimization. Technical report, 2021.
- Muñoz, M. A. and Smith-Miles, K. Generating new space-filling test instances for continuous black-box optimization. *Evolutionary computation*, 2020.
- Muñoz, M. A. and Smith-Miles, K. A. Performance analysis of continuous black-box optimization algorithms via footprints in instance space. *Evolutionary computation*, 25(4):529–554, 2017.
- Muñoz, M. A., Kirley, M., and Halgamuge, S. K. Exploratory landscape analysis of continuous space optimization problems using information content. *IEEE transactions on evolutionary computation*, 19(1):74–87, 2014.
- Novikov, A., Vű, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J., Mehrabian, A., et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Ono, Y., Harada, T., and Miura, Y. Large language model driven evolutionary optimization benchmark generation algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pp. 151–154, 2025.
- Pei, J., Mei, Y., Liu, J., and Zhang, M. Libog: Lifelong learning for black-box optimizer generation. *arXiv preprint arXiv:2505.13025*, 2025.
- Qian, Q., Huang, C., Xu, J., Lv, C., Wu, M., Liu, W., Wang, X., Wang, Z., Huang, Z., Tian, M., et al. Benchmark²: Systematic evaluation of llm benchmarks. *arXiv preprint arXiv:2601.03986*, 2026.
- Romera-Paredes, B., Barekatin, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J., Ellenberg, J. S., Wang, P., Fawzi, O., et al. Mathematical discoveries from

- program search with large language models. *Nature*, 625 (7995):468–475, 2024.
- Ros, R. and Hansen, N. A simple modification in cma-es achieving linear time and space complexity. In *International conference on parallel problem solving from nature*, 2008.
- Seiler, M. V., Kerschke, P., and Trautmann, H. Deep-ela: Deep exploratory landscape analysis with self-supervised pretrained transformers for single-and multi-objective continuous optimization problems. *Evolutionary Computation*, pp. 1–27, 2025.
- Shehadeh, M. A. and Kudela, J. Benchmarking global optimization techniques for unmanned aerial vehicle path planning, 2025. URL <https://arxiv.org/abs/2501.14503>.
- Skvorc, U., van Stein, N., Seiler, M., Grimme, B., Bäck, T., and Trautmann, H. Llm driven design of continuous optimization problems with controllable high-level properties. In *International Conference on the Applications of Evolutionary Computation (Part of EvoStar)*, pp. 183–199. Springer, 2026.
- Snoek, J., Larochelle, H., and Adams, R. P. Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25, 2012.
- Stanovov, V., Akhmedova, S., and Semenko, E. NI-shade-lbc algorithm with linear parameter adaptation bias change for cec 2022 numerical optimization. In *2022 IEEE CEC*, 2022.
- Storn, R. and Price, K. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 1997.
- Sun, J., Liu, X., Bäck, T., and Xu, Z. Learning adaptive differential evolution algorithm from optimization experiences by policy gradient. *IEEE TEVC*, 2021.
- Tao, X., Li, X., Chen, W., Liang, T., Li, Y., Guo, J., and Qi, L. Self-adaptive two roles hybrid learning strategies-based particle swarm optimization. *Information Sciences*, 578:457–481, 2021.
- Terayama, K., Sumita, M., Tamura, R., and Tsuda, K. Black-box optimization for automated discovery. *Accounts of Chemical Research*, 54(6):1334–1346, 2021.
- Thrun, S. and Pratt, L. Learning to learn: Introduction and overview. In *Learning to learn*. 1998.
- Tian, Y., Cheng, R., Zhang, X., and Jin, Y. Platemo: A matlab platform for evolutionary multi-objective optimization [educational forum]. *IEEE Computational Intelligence Magazine*, 12(4):73–87, 2017.
- Tian, Y., Peng, S., Zhang, X., Rodemann, T., Tan, K. C., and Jin, Y. A recommender system for metaheuristic algorithms for continuous optimization based on deep recurrent neural networks. *IEEE TAI*, 2020.
- Tian, Y., Qi, X., Yang, S., He, C., Tan, K. C., Jin, Y., and Zhang, X. A universal framework for automatically generating single-and multi-objective evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 2025.
- van Stein, N. and Bäck, T. Llamea: A large language model evolutionary algorithm for automatically generating metaheuristics. *IEEE Transactions on Evolutionary Computation*, 2024a.
- van Stein, N. and Bäck, T. Llamea: A large language model evolutionary algorithm for automatically generating metaheuristics. *IEEE TEVC*, 2024b.
- Vermetten, D., Ye, F., Bäck, T., and Doerr, C. Ma-bbob: Many-affine combinations of bbob functions for evaluating automl approaches in noiseless numerical black-box optimization contexts. In *International Conference on Automated Machine Learning*, pp. 7–1. PMLR, 2023.
- Wang, C., Gong, Y.-J., Cao, Z., and Ma, Z. Instance generation for meta-black-box optimization through latent space reverse engineering. *arXiv preprint arXiv:2509.15810*, 2025.
- Wei, J., Yang, Y., Zhang, X., Chen, Y., Zhuang, X., Gao, Z., Zhou, D., Wang, G., Gao, Z., Cao, J., et al. From ai for science to agentic science: A survey on autonomous scientific discovery. *arXiv preprint arXiv:2508.14111*, 2025.
- Wu, G., Mallipeddi, R., and Suganthan, P. N. Problem definitions and evaluation criteria for the cec 2017 competition on constrained real-parameter optimization. *National University of Defense Technology, Changsha, Hunan, PR China and Kyungpook National University, Daegu, South Korea and Nanyang Technological University, Singapore, Technical Report*, 9:2017, 2017.
- Wu, S.-H., Huang, Y., Wu, X., Feng, L., Zhan, Z.-H., and Tan, K. C. Learning to transfer for evolutionary multi-tasking. *IEEE Transactions on Cybernetics*, 2025.
- Xue, K., Chen, R.-T., Tan, R.-X., Lin, X., Shi, Y., Xu, S., Yuan, M., and Qian, C. Bboplace-bench: Benchmarking black-box optimization for chip placement. *arXiv preprint arXiv:2510.23472*, 2025.
- Yang, X., Wang, R., Li, K., and Ishibuchi, H. Meta-black-box optimization for evolutionary algorithms: Review

and perspective. *Swarm and Evolutionary Computation*, 93:101838, 2025.

Yao, S., Liu, F., Lin, X., Lu, Z., Wang, Z., and Zhang, Q. Multi-objective evolution of heuristic using large language model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 27144–27152, 2025.

Yazdani, D., Peng, M., Yazdani, D., Yazdi, S. F., Omidvar, M. N., Sun, Y., Nguyen, T. T., Li, C., and Li, X. Portal: Controllable landscape generator for continuous optimization-part i: Framework. *arXiv preprint arXiv:2512.00288*, 2025.

Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., and Song, G. Reevo: Large language models as hyper-heuristics with reflective evolution. *Advances in neural information processing systems*, 37:43571–43608, 2024.

Zhang, Q. and Li, H. Moea/d: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on evolutionary computation*, 11(6):712–731, 2007.

Zhang, Q., Zhou, A., Zhao, S., Suganthan, P. N., Liu, W., Tiwari, S., et al. Multiobjective optimization test instances for the cec 2009 special session and competition. 2008.

Zhao, Q., Liu, T., Yan, B., Duan, Q., Yang, J., and Shi, Y. Automated metaheuristic algorithm design with autoregressive learning. *IEEE TEVC*, 2024.

Zhong, Y., Wang, X., Sun, Y., and Gong, Y.-J. Sddobench: A benchmark for streaming data-driven optimization with concept drift. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 59–67, 2024.

System Prompt (Part 1)

You are an expert Python/NumPy programmer specializing in symbolic mathematical function construction.
Your goal is to generate concise, vectorized, and syntactically correct Python code.

Behavioral Guidelines:

- Ensure the generated code runs without syntax errors.
- Avoid imports beyond `import numpy as np`.
- Use pure numpy-based vectorized operations only.
- Follow the following given structural constraints strictly (e.g., class names, method signatures, etc.):

```
python
class GeneratedFunction:
def __init__(self, dim = {self.dim}, ub = 5., lb = -5.):
self.dim = dim
self.ub = ub
self.lb = lb
def func(self, x):
# vectorized expression here
res = ...
return res
```

- Always ensure the generated class name is `'GeneratedFunction'`.
- `'__init__(self, dim, ub, lb)'` — stores the dimensionality and `x`'s bounds.
- `'func(self, x)'` — accepts `'x'` as a numpy array of shape `(num_of_x, dim)`.
- Output a numpy array `'res'` of shape `(num_of_x,)`.
- Avoid mixing scalar and row vectors directly (e.g., `x[:, 0] + np.sin(x)` is invalid).
- Always reduce across columns using `np.sum(..., axis=1)` or `np.mean(..., axis=1)` when combining components.
- Avoid mixing `(n,)` and `(n,d)` shapes directly.
- `'axis = 0'` is FORBIDDEN in this context.
- Always output ONLY valid Python code — no explanations or comments.

...(The content below is in Part 2)

System Prompt (Part 2)

Code Style Constraints (Strictly Enforced)

1. **Parsimony Principle**: The best solution is the simplest one. Do NOT create "spaghetti code" by stacking random operators.

2. **Functional Composition**: Prefer nested functional calls over defining many intermediate variables.

```
python
a = np.sin(x)
b = np.tanh(a)
c = np.exp(b)
```

```
python
c = np.exp(np.tanh(np.sin(x)))
```

3. **Variable Limit**: You are allowed a MAXIMUM of 5-8 intermediate variables. Variables should only be defined if they represent a distinct, reusable physical concept (e.g., `'amplitude'`, `'frequency'`, `'decay'`).

4. **Vectorization**: Use NumPy vectorization efficiently. Avoid creating variables `'A'`, `'B'`, `'C'` just to sum them up at the end.

Figure 6. Code Generation Guideline (System Prompt)

A. Prompts in EoB

A.1. Code Generation Guideline (System Prompt)

This prompt serves as the foundational rule set for the LLM, strictly regulating the format, interface, and stylistic quality of the generated Python code. The detailed design of the prompts is shown in fig 6. Each part of the prompt and its description will be highlighted in three different colors, as described below:

- **Interface Standardization**: To ensure the generation of executable and mathematically coherent benchmarks, this prompt conditions the LLM as a specialized NumPy expert, enforcing a strict schema where outputs must be encapsulated within a `'GeneratedFunction'` class supporting vectorized inputs. Crucially, the prompt imposes rigorous syntactic and stylistic regularization to guarantee solvability and readability: it mandates shape consistency (e.g., prohibiting reduction along the batch dimension to preserve sample independence).
- **Parsimony Principle**: To prevent the generated function code from becoming "spaghetti-like," our prompt design requires the LLM to avoid defining a large number of intermediate variables when generating nested structured functions. Instead, it should construct them by directly combining functions.
- **Concise, Efficient, and Meaningful**: We restrict the number of intermediate variables generated by the LLM during the code generation process, requiring each intermediate variable to correspond to a specific physical concept to ensure meaningfulness. Additionally, we mandate the efficient use of NumPy operations during code generation to improve performance.

A.2. Initialization

In the strategy described in Section 3.2.1, we employ a carefully constructed *Init Prompt* to help LLM grasps the essential context information, such as its role (as an optimization expert and benchmark designer) and its specific task (generating program code for the objective function). The knowledge regarding the code generation of the objective function in the

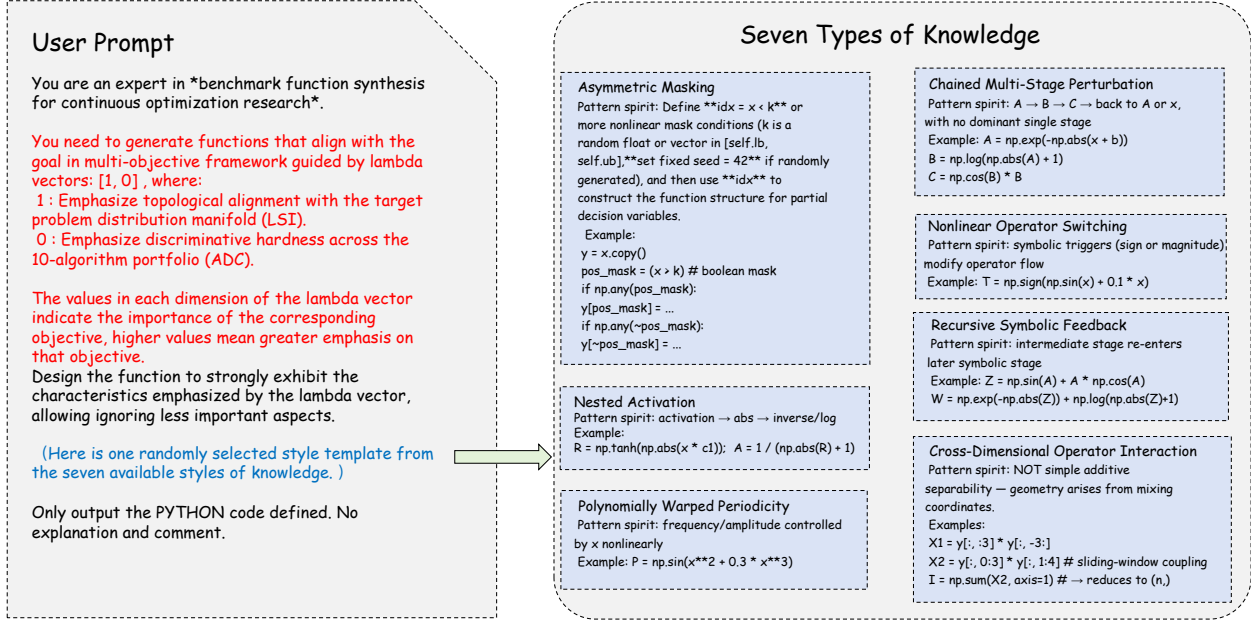


Figure 7. Initialization Prompt (User)

Init_Prompt is provided by the *System_Prompt* in Appendix A.1, while the knowledge related to the roles of the LLM as a benchmark designer and optimization expert is provided by the *User_Prompt*. The content of the *User_Prompt* is shown in Fig 7.

The *User_Prompt* consists of two major parts, which are highlighted in different colors in the following content and in Fig 7 :

- **Generation Preference.** This part drives the generation of the initial individual by explicitly defining the multi-objective optimization context. The prompt explicitly maps the reference vector $\vec{\lambda}_i = [\lambda_{i,1}, \lambda_{i,2}]$ dimensions to semantic objectives: $\lambda_{i,1}$: Emphasizes topological alignment with the target problem distribution (LSI); $\lambda_{i,2}$: Emphasizes discriminative hardness, encouraging the creation of "strategy-breaker" landscapes that differentiate between algorithms (ADC). Guided by this part, LLM can design the function that strongly exhibits the characteristics emphasized by $\vec{\lambda}_i$, while ignoring less important aspects.
- **Population Diversity.** To address the issue of LLM generating a large number of highly similar function instances due to a lack of landscape knowledge, which leads to a loss of diversity in the initial population, we constructed a knowledge base comprising seven types of landscape styles (as shown in the right part of Fig 7). By injecting random landscape style knowledge into the prompts used to generate initial individuals, we enhance both the code diversity and landscape diversity of the initial population. The seven types of landscape style knowledge are introduced as follows:
 - **Asymmetric Masking.** The Asymmetric Masking style explicitly disrupts global symmetry by applying distinct symbolic operations to sub-regions of the search space defined by boolean conditions (e.g., $x < k$). By forcing the LLM to use `np.where` or boolean indexing, this style creates landscapes with sharp regional shifts and discontinuities, challenging algorithms that assume global rotational invariance.
 - **Nested Activation.** The Nested Activation style (often implemented via rational composition) creates landscapes with steep, controlled basins. It enforces a specific operator pipeline, typically composing activation functions with rational transformations (e.g., $f(x) = \frac{1}{|\tanh(x)|+1}$) to generate narrow valleys that test algorithmic stability without causing numerical overflow.
 - **Polynomically Warped Periodicity.** The Polynomically Warped Periodicity style modulates the frequency or amplitude of periodic functions using nonlinear polynomial terms. Unlike standard Rastrigin-like functions, this style produces "chirp-like" structures (e.g., $\sin(x^2 + 0.3x^3)$) where the gradient frequency changes dynamically across the search space, challenging step-size adaptation mechanisms.

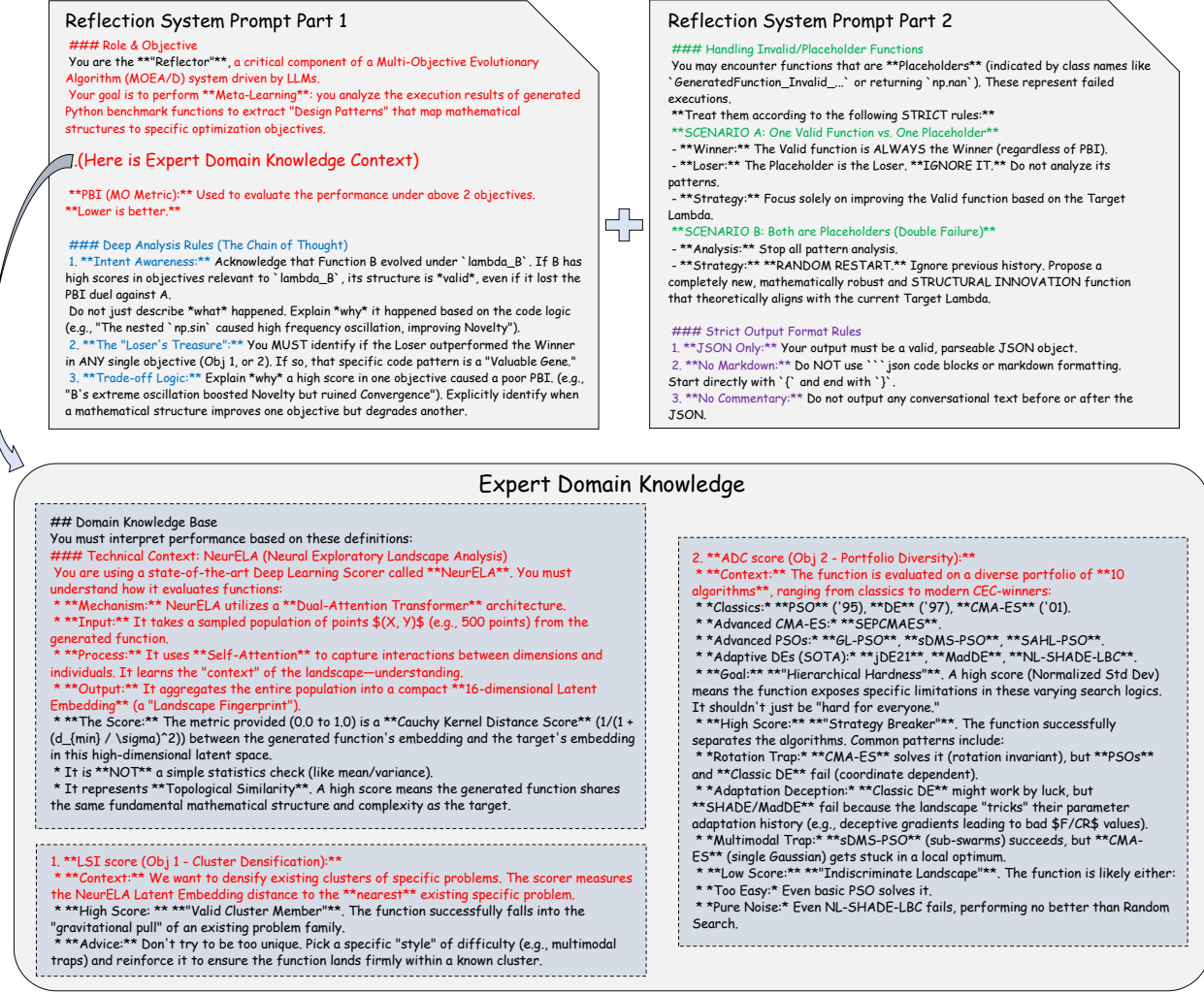


Figure 8. Reflection Stage System Prompt

- **Chained Multi-stage Perturbation.** The Chained Multi-stage Perturbation style fosters feature mixing by enforcing a cyclical composition of disparate mathematical operators (e.g., exponential decay feeding into logarithmic scaling, then into trigonometric oscillation). This ensures that no single operator dominates the global trend, resulting in hybrid topologies that are difficult to classify under standard BBOB categories.
- **Recursive Symbolic Feedback.** The Recursive Symbolic Feedback style generates fractal-like micro-structures by allowing intermediate symbolic variables to re-enter calculations later in the pipeline (e.g., $Z = \sin(A) + A \cdot \cos(A)$). This recursion creates deep variable dependencies where small input changes propagate into high-frequency local optima.
- **Nonlinear Operator Switching.** The Nonlinear Operator Switching style utilizes symbolic triggers (such as magnitude or sign) to modify operator flow, creating "manifold switching" effects where the landscape topology changes smoothly but drastically.
- **Cross-Dimensional Operator Interaction** The Cross-dimensional Operator Interaction module mandates coordinate mixing (e.g., sliding window coupling or product interactions like $x_i \cdot x_j$ and $x[k] * x[-k]$). Uniquely, this style permits temporary dimensional reduction (e.g., `np.sum` along the feature axis) to create interaction terms, which are then broadcast back to the original shape, thereby invalidating algorithms that rely on independent coordinate search.

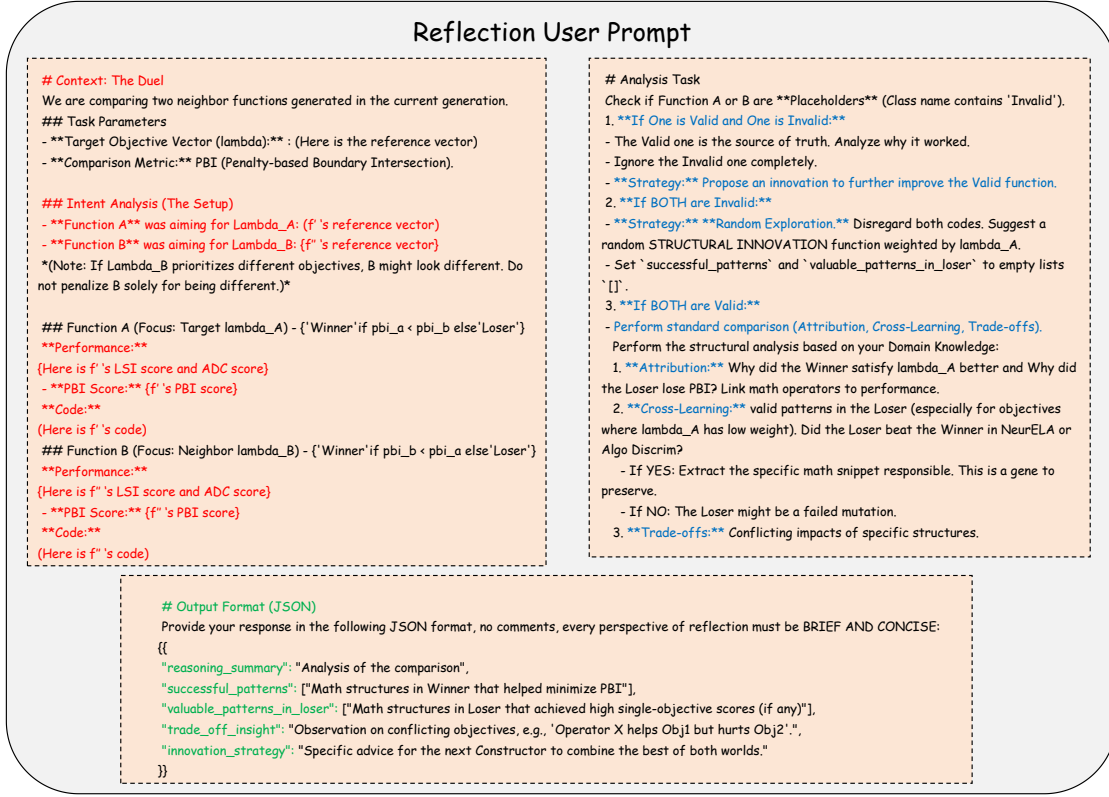


Figure 9. Reflection User Prompt

In conclusion, the *Init.Prompt* relies on a synergistic prompting strategy that integrates the *System.Prompt* and the *User.Prompt*. The *System.Prompt* acts as a strict Code Generation Guideline, imposing strict syntactic constraints, interface standardization, and stylistic regularization to guarantee the executability and numerical stability of the generated code. The *User.Prompt* injects the multi-objective reference vectors ($\vec{\lambda}$) and specific Landscape Construction Knowledge that randomly selected from the seven landscape styles detailed above. This dual-prompt framework effectively conditions the LLM to function as a domain expert, ensuring that the resulting initial population comprises objective function programs that are not only computationally compliant but also exhibit rich and distinct diversity, laying a robust foundation for the subsequent evolutionary process.

A.3. Reflection & Reproduction

As mentioned in Section 3.2.3, the reflection and reproduction stages serve as the cognitive core of our evolutionary framework, simulating the crossover and mutation mechanisms of traditional evolutionary algorithms but operating within the semantic space of code analysis. The core objective is to convert numerical multi-objective performance into actionable coding insights.

A.3.1. REFLECTION STAGE

To enable the LLM to perform high-level reasoning and comparative analysis, we designed a two-part *Reflection.Prompt* that explicitly instructs the LLM to play the role of a senior optimization and landscape design expert. This enables LLM to interpret complex code structures based on the two objectives defined in Section 3.2.2, as well as knowledge of landscape theory and optimization principles. The *Reflection.Prompt* consists of the following two components:

- **System Prompt.** The system prompt serves as the foundational rule set for the Reflection stage. It transforms raw execution program into actionable design intelligence by analyzing "Winner vs. Loser" pairs to extract causal links between mathematical structures and multi-objective metrics. The prompt consists of four streamlined modules which

highlighted with different color in Fig 8 and following content :

- **Role & Objective (with Domain Knowledge).** This module establishes the role of an expert "Reflector" and injects a specialized Domain Knowledge Base. It provides theoretical context for NeurELA's structure and mechanism, LSI score and ADC score, ensuring all analysis is technically grounded.
 - **Deep Analysis Rules.** This module requires the LLM to conduct a three-fold in-depth reflection (when comparing functions f' and f'' , a candidate is labeled as the winner if its PBI is higher, or as the loser otherwise): 1) Successful Pattern: Identify the major code-level differences between the winner and the loser, summarize such knowledge, and relate it to landscape characteristics. 2) Gene Preservation: If the loser scores higher in either LSI or ADC, the LLM is asked to analyze the reasons behind it and preserve such valuable knowledge. 3) Bi-objective Tradeoff: Recognize and analyze the positive/negative impacts of certain parts of the program's code on LSI and ADC, with a special focus on conflict cases—for example, adding a mathematical operation may increase LSI but harm ADC. Such cases serve as the basis for crossover operations.
 - **Handling Invalid Functions.** This module ensures robustness in runtime errors by defining strict branching logic. In the scenario of "one valid item and one placeholder item," it focuses solely on improving the valid function. In the scenario of "dual failure," it triggers a "random restart," instructing the model to abandon the current path and propose entirely new structural innovations. Such cases serve as the basis for mutation operations.
 - **Output Formatting Constraints.** To facilitate automated processing, this module enforces a rigid JSON-only output format. It strictly forbids Markdown formatting or conversational fillers, ensuring the extracted insights and improvement strategies can be directly parsed and utilized by the subsequent Reproduction stage.
- **User Prompt.** The user prompt serves as a dynamic context injector, incorporating details about the selected two candidates, f' and f'' , from the neighborhood set into a templated prompt. It dynamically constructs a task prompt for each function evolution duel. The user prompt can be divided into the following three parts, with the content of each section highlighted in different colors in the text below and in Fig 9:
- **Contextual Instantiation (The Duel Setup).** This module provides the raw material for analysis by injecting the source codes, LSI, ADC, and PBI Scores of the paired candidates f' and f'' . Crucially, it explicitly provides the specific reference vector ($\vec{\lambda}'_f$) and the neighbor's vector ($\vec{\lambda}''_f$). This enables the LLM to integrate the preferences of its reference vector when conducting comparative analysis of a pair of functions, thereby extracting insights and experience. This prevents the LLM from relying solely on absolute LSI or ADC scores and drawing extreme evolutionary insights.
 - **Scenario-Based Evolution.** This module explicitly marks the validity status of each function (identifying the "Invalid" category), thereby requiring the LLM to provide evolutionary insights in different directions. The design of this process is similar to the corresponding section in the reflection system prompt mentioned above. It instructs the LLM to perform the following operations: conduct standard comparative analysis when both functions are valid; execute "innovative improvement" guided by the reference vector when only a single valid function remains; or trigger "random exploration" in cases of dual failure.
 - **Strategy Output Format.** This module requires mapping the final analysis to the specified JSON output fields. The content of the JSON fields is largely consistent with the analytical rules of the reflection system prompt. It instructs the LLM to dissect the "winner" to extract successful patterns (i.e., structures that minimize the PBI), output as "successful patterns"; to mine valuable patterns from the "loser", specifically, certain code snippets that may score high on a single objective (such as high ADC) but perform poorly overall in terms of PBI, output as "valuable patterns in loser"; to analyze code snippets where potential improvement conflicts exist, output as "trade off insight"; and finally, to output the summarized insights as "innovation strategy".

After completing the design of the aforementioned system prompt and user prompt, during the evolution process, the reflector dynamically injects into the user prompt the source code, LSI, ADC, and PBI scores required for reflection, along with the corresponding reference vector. It then combines the two prompts and calls the LLM to perform all intermediate analyses and reasoning, organizing the response content in a JSON-like format. This includes a summary of the overall reasoning process, condensed analysis results for the above case-by-case reflection, and suggestions/instructions for generating the next generation offspring program.

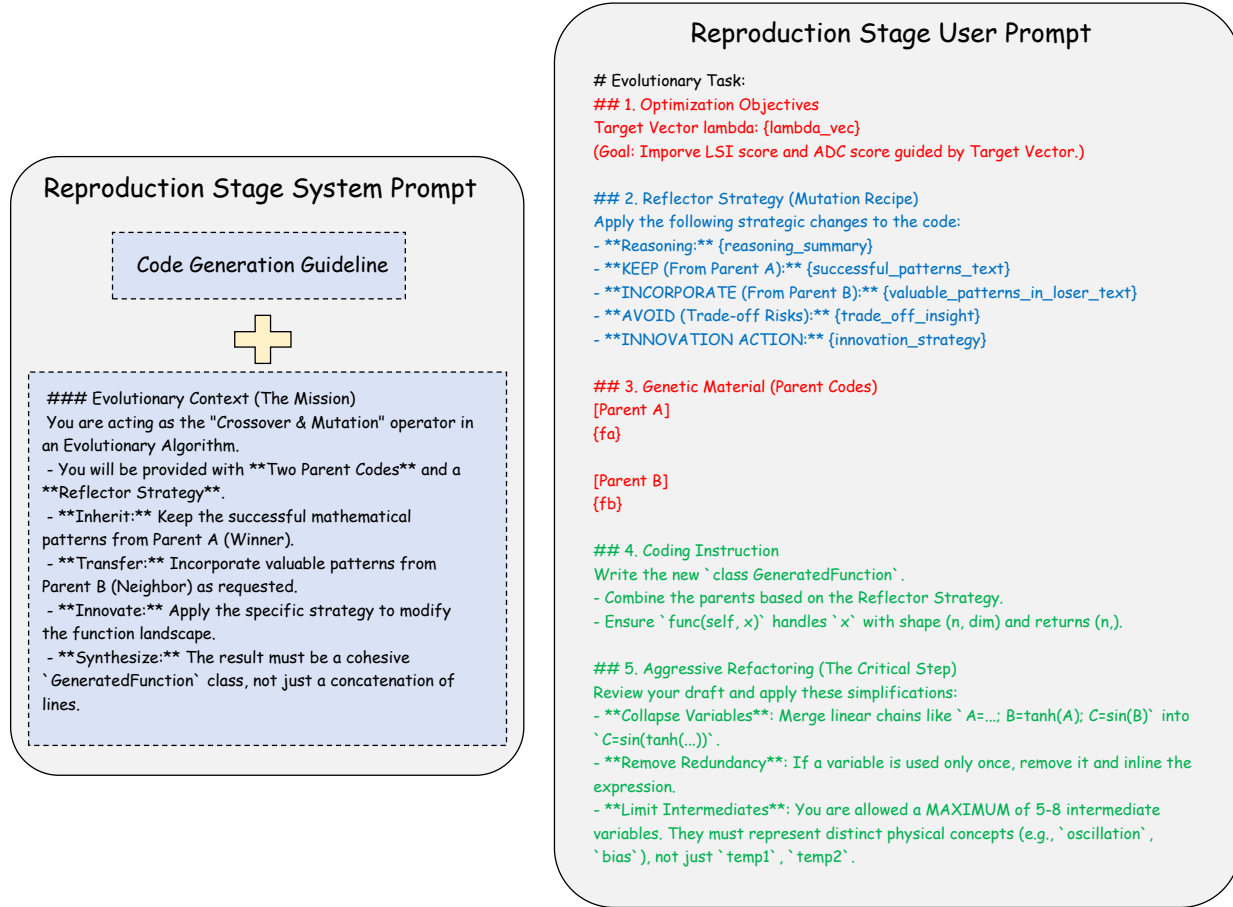


Figure 10. Reproduction Stage Prompt

A.3.2. REPRODUCTION STAGE

Based on the JSON-like reflections obtained in the reflection stage, the reproduction stage requires the LLM to refer to and follow these useful analyses or improvement suggestions to generate new program candidates. In this stage, a new role must be defined for the LLM, and the JSON-formatted reflections generated in the reflection stage need to be unpacked and injected into a prompt template that includes the reference vector and the codes of the two parent. Similarly, the prompt for this stage also consists of two parts, as illustrated in Fig 10 and described below:

- **System Prompt.** In addition to strictly adhering to the code generation guidelines defined in Appendix A.1, the system prompt for the reproduction stage clearly defines its role as an evolutionary operator (crossover and mutation). This module, guided by a specific reflection strategy, enables the LLM to process two parent codes and synthesize a new offspring. The generation process follows a strict four-step protocol: inheritance (retain the robust mathematical structure of the winner), transfer (assimilate beneficial "genes" from the neighbor), innovation (apply strategic perturbations), and synthesis. Crucially, the prompt requires the final output to be a unified, mathematically coherent `GeneratedFunction` class, explicitly prohibiting the mechanical concatenation of unrelated code segments.
- **User Prompt.** The user prompt for the reproduction stage is dynamically constructed by parsing and unpacking the structured JSON insights derived from the Reflection stage, while simultaneously injecting the source codes of the two parent candidates and target reference vector. This prompt is structurally organized into three distinct operational modules highlighted in different colors in Fig 10 and following content:
 - **Optimization Context and Genetic Material.** This module establishes the boundary conditions for the new generation. It explicitly defines the Optimization Objectives (including the target reference vector $\vec{\lambda}$ and specific

landscape goals) and provides the raw Genetic Material by embedding the full source codes of two parent (f_A and f_B). This ensures the LLM has direct access to both the specific directional goal and the exact syntax of the predecessor functions.

- **The Reflector Strategy.** Acting as the core evolutionary directive, this module translates the insights extracted from the previous stage into actionable instructions. It presents a structured "Evolution Recipe" that explicitly dictates: 1) Keep: Which successful patterns from Parent A must be preserved. 2) Incorporate: Which specific "genes" (code snippets) from Parent B should be transferred. 3) Avoid: Which trade-off risks to mitigate. 4) Innovation: The specific landscape modification strategy to apply. This ensures that the crossover is not random, but a directed synthesis based on the Reflector's analysis.
- **Coding Instruction and Aggressive Refactoring.** This module enforces strict implementation standards to maintain code quality. The rules for code writing and refactoring in the prompt are similar to those in the code generation guidelines of the system prompt and will not be elaborated on further here.

B. Parameter Sensitivity Results

B.1. Search Dimension D

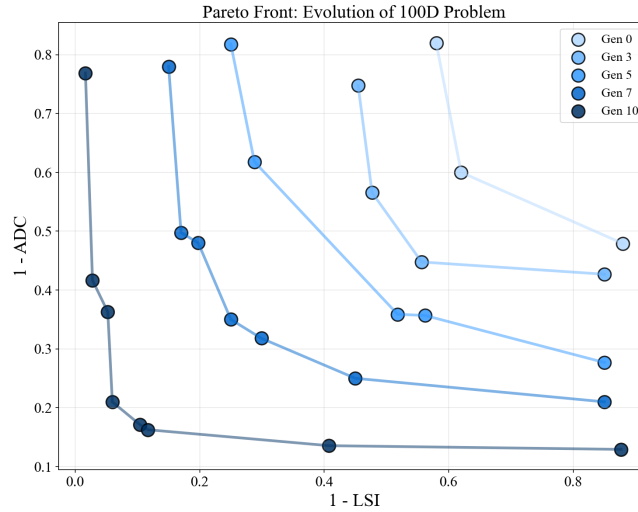


Figure 11. Pareto front evolution of EoB under the 100D setting.

Figure 11 shows the Pareto evolution under the 100D setting. The evolving fronts progressively move toward better landscape similarity and stronger algorithm distinguishing capability, suggesting that EoB remains effective in high-dimensional benchmark synthesis.

B.2. Population Size N

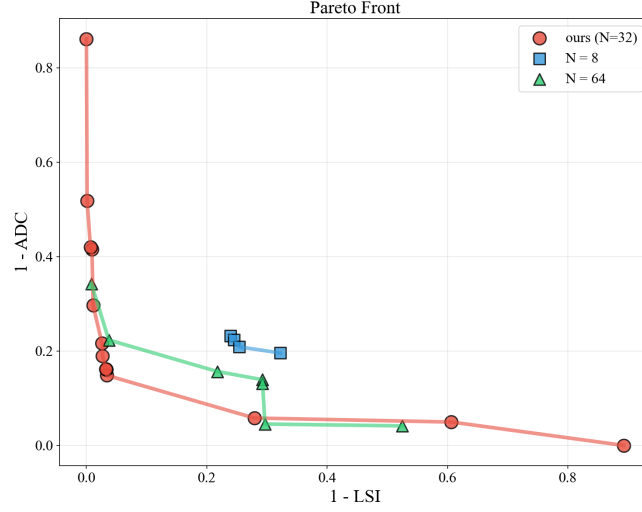


Figure 12. Pareto fronts obtained by EoB with different population sizes.

Figure 12 compares the Pareto fronts produced by different population sizes. The small setting $N = 8$ leads to a narrow front concentrated in the middle trade-off region, while $N = 64$ does not provide a clear advantage over $N = 32$ and incurs higher cost.

B.3. Target Task Set Size H

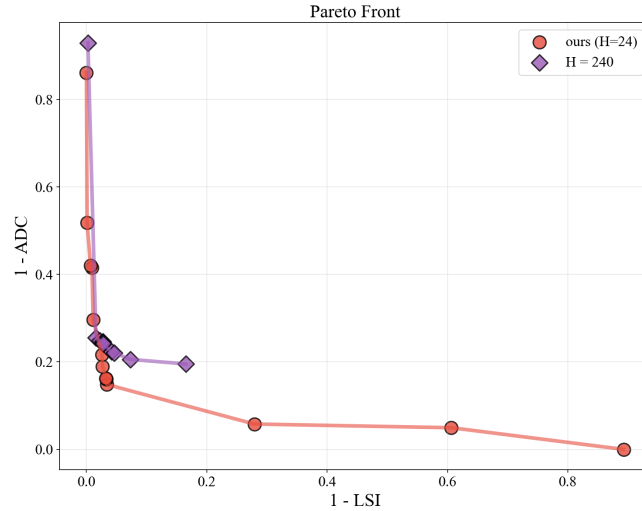


Figure 13. Pareto front comparison between the default target task set size and $H = 240$.

Figure 13 compares the default target task set size with $H = 240$. The two fronts exhibit similar behavior, indicating that EoB is not sensitive to the number of target task instances.